Supplementary Material for Full-Gradient Successor Feature Learning for Multi-Task Transfer in Reinforcement Learning

Ritish Shrirao
Dept. of CSE, IIIT Bangalore
ritish.shrirao@iiitb.ac.in

Aditya Priyadarshi
Dept. of CSE, IIIT Bangalore
aditya.priyadarshi@iiitb.ac.in

Raghuram Bharadwaj
Dept. of DSAI, IIIT Bangalore
raghuram.bharadwaj@iiitb.ac.in

This document contains the appendices of the paper. Numbers of sections, equations, figures, tables and results continue those of the main paper.

Appendix A: Proofs of the Lemmas

Throughout, $\mathcal{F}_k$ is as in Assumption 2, $\omega_k = (i_k, s_k, a_k, \mathcal{K}_k)$ is the sample used at iteration k, and ties are broken by the fixed deterministic rule of Section IV, so that $\tilde{G}_k = G(\Theta_k, \omega_k)$ for a measurable map G. For $R > 0$ let

$$K_{1,R} := \max_{j,s,a,\phi} \sup_{\|\theta\| \le R} |\xi_j(s, a, \phi; \theta)|,$$
$$K_{2,R} := \max_{j,s,a,\phi} \sup_{\|\theta\| \le R} \|\nabla_\theta \xi_j(s, a, \phi; \theta)\|,$$

which are finite by Assumption 1. They bound the ξ-values and their gradients while the parameters stay in the ball of radius R.

Proof of Lemma 1(c): We follow [1, Prop. 3.27] but use V only on L.

Setting. Recall [1, Sec. 3.3] that a compact set L is internally chain transitive if for all $x, y \in L$, $\varepsilon > 0$ and $T > 0$ there are solutions $x_1, \dots, x_n$ of $\dot{x} \in F(x)$ and times $t_1, \dots, t_n > T$ such that

- $x_i([0, t_i]) \subseteq L$ for all i;
- $\|x_i(t_i) - x_{i+1}(0)\| \le \varepsilon$ for $i < n$;
- $\|x_1(0) - x\| \le \varepsilon$ and $\|x_n(t_n) - y\| \le \varepsilon$.

We call this an (ε, T)-chain from x to y. We use two facts:

(F1) **V is nonincreasing along every solution segment in L**, since a time-shifted solution is again a solution and the hypothesis applies to its sub-segments;

(F2) **the solutions with initial points in a compact set form a compact set of paths**, with the topology of uniform convergence on compact time intervals [1, Lemma 3.1].

Step 1: V is constant on L. Intuitively, a chain cannot climb from the minimum to the maximum of V, since V decreases along each segment and the jumps are small. Since L is compact and V is continuous on L, $v = \min_L V$ and $\bar{v} = \max_L V$ are attained. Suppose, for contradiction, that $v < \bar{v}$. Since $V(\Lambda)$ has empty interior, we can choose a level

$$u \in (v, \bar{v}) \quad \text{with} \quad u \notin V(\Lambda).$$

Step 1a: segments in L cross the level u with a margin.

Claim. There is $\kappa > 0$ such that every solution z satisfies

$$\left.\begin{array}{l} z([0,t]) \subseteq L,\ t \ge 1, \\ V(z(0)) \le u + \kappa \end{array}\right\} \implies V(z(t)) \le u - \kappa.$$

That is, a segment of length at least 1 that starts near the level u ends clearly below it.

Proof of the claim. Suppose not. Then for every n there are z_n and $t_n \ge 1$ with $z_n([0, t_n]) \subseteq L$, $V(z_n(0)) \le u + 1/n$ and $V(z_n(t_n)) > u - 1/n$. By (F1),

$$u - 1/n < V(z_n(1)) \le V(z_n(0)) \le u + 1/n.$$

By (F2), a subsequence converges uniformly on $[0, 1]$ to a solution z. Since L is closed and V is continuous on L, $z([0,1]) \subseteq L$ and $V(z(0)) = V(z(1)) = u$. But $u \notin V(\Lambda)$, so $z(0) \notin \Lambda$, and the hypothesis gives $V(z(1)) < V(z(0))$, a contradiction. ◁

Step 1b: no chain climbs from v to $\bar{v}$. Let

$$\eta(\varepsilon) = \sup\{|V(p) - V(q)| : p, q \in L,\ \|p - q\| \le \varepsilon\},$$

which tends to 0 as $\varepsilon \to 0$ because V is uniformly continuous on the compact set L. Choose $x, y \in L$ with $V(x) = v$ and $V(y) = \bar{v}$, choose ε with $\eta(\varepsilon) < \min(\kappa, u - v)$, and take an $(\varepsilon, 1)$-chain from x to y. All points $x_i(0)$ and $x_i(t_i)$ lie in L, so each jump of the chain changes V by at most $\eta(\varepsilon)$. By induction, every segment of the chain ends at or below $u - \kappa$:

- the chain starts below u: $V(x_1(0)) \le v + \eta(\varepsilon) < u$;
- if $V(x_i(0)) \le u + \kappa$, Step 1a gives $V(x_i(t_i)) \le u - \kappa$, hence $V(x_{i+1}(0)) \le u - \kappa + \eta(\varepsilon) < u$.

In particular $V(x_n(t_n)) \le u - \kappa$, so

$$\bar{v} = V(y) \le u - \kappa + \eta(\varepsilon) < u < \bar{v},$$

a contradiction. Hence V is constant on L.

Step 2: $L \subseteq \Lambda$. Let $x \in L$. By Lemma 1(b), a solution through x stays in L. If $x \notin \Lambda$, V decreases strictly along it, which is impossible since V is constant on L. Hence $x \in \Lambda$, so $L \subseteq \Lambda$. □

Lemma 2 (Martingale difference noise)**.** *Let $h^{(N)}(\Theta) := \mathbb{E}_\omega[G(\Theta, \omega)]$, where ω has the law of Assumption 2 with batch size N, and let $M_{k+1} := \tilde{G}_k - h^{(N_k)}(\Theta_k)$. Then:*

1) $\mathbb{E}[M_{k+1} \mid \mathcal{F}_k] = \mathbf{0}$ *a.s. for all $k \ge 0$;*

2) *for every $R > 0$, on $\{\|\Theta_k\| \le R\}$,*

$$\mathbb{E}\big[\|M_{k+1}\|^2 \mid \mathcal{F}_k\big] \le C_R := 32|\Phi|^2(1+2K_{1,R})^2K_{2,R}^2.$$

Proof. **Part 1: martingale difference property.** Given $\mathcal{F}_k$, the only randomness in $\tilde{G}_k$ is the fresh sample ω_k. Θ_k is $\mathcal{F}_k$-measurable, and by Assumption 2 the conditional law of ω_k given $\mathcal{F}_k$ is the fixed sampling law with batch size N_k. Hence $\mathbb{E}[\tilde{G}_k \mid \mathcal{F}_k] = \mathbb{E}_\omega[G(\Theta,\omega)]\big|_{\Theta=\Theta_k} = h^{(N_k)}(\Theta_k)$, which is $\mathcal{F}_k$-measurable, **and $\mathbb{E}[M_{k+1} \mid \mathcal{F}_k] = 0$.**

Part 2: bounded conditional second moments. It suffices to bound the size of the update itself. Since $h^{(N_k)}(\Theta_k) = \mathbb{E}[\tilde{G}_k \mid \mathcal{F}_k]$,

$$\begin{aligned}\mathbb{E}\big[\|M_{k+1}\|^2 \mid \mathcal{F}_k\big] &= \mathbb{E}\big[\|\tilde{G}_k\|^2 \mid \mathcal{F}_k\big] - \|h^{(N_k)}(\Theta_k)\|^2 \\ &\le \mathbb{E}\big[\|\tilde{G}_k\|^2 \mid \mathcal{F}_k\big]. \end{aligned} \tag{19}$$

For an active block $j \in \{i_k, c_k\}$, write (10) as $\tilde{g}_j = 2\sum_\phi \bar{\delta}_j(\phi)\,\bar{D}_j(\phi)$, with $\bar{D}_j(\phi)$ as in Section V. On $\{\|\Theta_k\| \le R\}$, since averages obey the same bounds as their summands and $\gamma_p \le 1$,

$$|\bar{\delta}_j(\phi)| \le 1 + 2K_{1,R}, \qquad \|\bar{D}_j(\phi)\| \le 2K_{2,R}. \tag{20}$$

Hence, by the triangle inequality,

$$\|\tilde{g}_j\| \le 2|\Phi|(1+2K_{1,R})(2K_{2,R}) = 4|\Phi|(1+2K_{1,R})K_{2,R}. \tag{21}$$

The blocks of $\tilde{G}_k$ are disjoint and at most two are non-zero, so

$$\begin{aligned}\|\tilde{G}_k\|^2 &\le 2 \max_{j\in\{i_k,c_k\}} \|\tilde{g}_j\|^2 \\ &\le 32|\Phi|^2(1+2K_{1,R})^2K_{2,R}^2 = C_R. \end{aligned} \tag{22}$$

Taking conditional expectations completes the proof. □

Lemma 3 (Mean-field identification and batch bias)**.** *Under Assumptions 1 and 2, let $h^{(N)}$ be as in Lemma 2, and let $h^{(\infty)}(\Theta)$ be the population field obtained by replacing the sample averages in* (8)–(10) *with conditional expectations under $P(\cdot \mid s,a)$, using the same tie-breaking rule. Then:*

(a) *$\mathbb{E}[\tilde{G}_k \mid \mathcal{F}_k] = h^{(N_k)}(\Theta_k)$, with $h^{(N)}(\Theta) \in \overline{H}^{(N)}(\Theta)$ and $h^{(\infty)}(\Theta) \in \overline{H}^{(\infty)}(\Theta)$ for all Θ;*

(b) *for all Θ, $E^{(N)}(\Theta) = E^{\mathrm{BE}}(\Theta) + V(\Theta)/N$ (Proposition 1). Moreover, $E^{(N)}, E^{\mathrm{BE}}, V \in C^2(\mathcal{U})$, and on $\mathcal{U}$ we have $\nabla E^{(N)} = h^{(N)}$ and $\nabla E^{\mathrm{BE}} = h^{(\infty)}$, so that $\overline{H}^{(N)} = \{\nabla E^{(N)}\}$ and $\overline{H}^{(\infty)} = \{\nabla E^{\mathrm{BE}}\}$ there;*

(c) *for every $R > 0$ and $N \ge 1$, $\sup_{\|\Theta\|\le R}\|h^{(N)}(\Theta) - h^{(\infty)}(\Theta)\| \le C'_R/N$, where $C'_R := 16|\Phi|(1 + K_{1,R})K_{2,R}$.*

Proof. **Step 1: conditioning.** Part (a) for $h^{(N_k)}$ is Part 1 of the proof of Lemma 2. For every ω, $G(\Theta,\omega) = G^\sigma(\Theta,\omega)$ for the selection $\sigma \in \Sigma(\Theta,\omega)$ produced by the tie-breaking rule. Hence $G(\Theta,\omega) \in F^{(N)}(\Theta,\omega)$, and taking expectations gives $h^{(N)}(\Theta) \in \overline{H}^{(N)}(\Theta)$. The same argument applies to $h^{(\infty)}$.

Step 2: covariance identity. The averaged update multiplies two sample means, and the mean of their product differs from the product of their means by a covariance divided by N. Fix (i,s,a), Θ, an active block $j \in \{i,c\}$ with $c = c(\Theta,s,i)$, and $\phi \in \Phi$. Let $Y_p := \mathbb{I}(\phi = \phi_p) + \gamma_p\,\xi_j(s'_p, b_{j,p}, \phi; \theta^{(j)})$ and $D_p := \gamma_p \nabla\xi_j(s'_p, b_{j,p}, \phi; \theta^{(j)})$, and let $x := \xi_j(s,a,\phi;\theta^{(j)})$ and $y := \nabla\xi_j(s,a,\phi;\theta^{(j)})$. The prior c depends only on (Θ,s,i), and $b_{j,p} = b_j(\Theta,s'_p)$ depends only on (Θ,s'_p). Hence the pairs (Y_p, D_p) are i.i.d. given (i,s,a), with means (m_Y, m_D). For the sample means $\bar{Y}$ and $\bar{D}$,

$$\begin{aligned}\mathbb{E}\big[(\bar{Y}-x)(\bar{D}-y)\big] &= (m_Y - x)(m_D - y) \\ &\quad + \tfrac{1}{N}\mathrm{Cov}(Y_1, D_1), \end{aligned} \tag{23}$$

$$\mathbb{E}\big[(\bar{Y}-x)^2\big] = (m_Y - x)^2 + \tfrac{1}{N}\mathrm{Var}(Y_1). \tag{24}$$

The first terms on the right are the population quantities, since

$$\begin{aligned}m_Y = \mathbb{E}_{s'\sim P(\cdot|s,a)}\big[&\mathbb{I}(\phi = \phi(s,a,s')) \\ &+ \gamma(s')\,\xi_j(s', b_j(\Theta,s'), \phi; \theta^{(j)})\big]\end{aligned}$$

is the expected Bellman target of block j, which replaces the per-sample target in E^{BE}.

Step 3: decomposition and gradient. Summing (24) over ϕ and over the active blocks and taking $\mathbb{E}_{i\sim\rho}\mathbb{E}_{(s,a)\sim\mu_i}$ gives $E^{(N)} = E^{\mathrm{BE}} + V/N$ for every Θ. Fix $\Theta \in \mathcal{U}$. On a neighborhood of Θ, all selections c and $b_j(\cdot)$ are constant. Because the spaces are finite, $E^{(N)}$ is there a finite sum of C^2 functions of Θ, which may be differentiated term by term. Moreover, ℓ_i depends only on $\theta^{(i)}$ and ℓ_c only on $\theta^{(c)}$. Hence $\nabla_{\theta^{(j)}} E^{(N)} = \mathbb{E}[G_j]$, i.e., $\nabla E^{(N)} = h^{(N)}$. The same argument applied to E^{BE} gives $\nabla E^{\mathrm{BE}} = h^{(\infty)}$. Since Σ is a singleton near Θ, $\overline{H}^{(N)}(\Theta) = \{h^{(N)}(\Theta)\}$ and $\overline{H}^{(\infty)}(\Theta) = \{h^{(\infty)}(\Theta)\}$.

Step 4: batch bias. By (23), the block-j components of $h^{(N)}$ and $h^{(\infty)}$ differ by $\mathbb{E}_{i,s,a}\big[\frac{2}{N}\sum_\phi \mathrm{Cov}(Y_1, D_1)\big]$. On $\{\|\Theta\| \le R\}$ we have $|Y_1 - m_Y| \le 2(1+K_{1,R})$ and $\|D_1 - m_D\| \le 2K_{2,R}$, so $\|\mathrm{Cov}(Y_1,D_1)\| \le 4(1+K_{1,R})K_{2,R}$. Summing over $|\Phi|$ features and at most two blocks yields (c). In particular, the bias is $O(1/N)$ rather than $O(N^{-1/2})$: the $O(N^{-1/2})$ fluctuations of $\bar{Y}$ and $\bar{D}$ have conditional mean zero and enter M_{k+1}, and only their covariance shifts the mean field. □

Remark 1 (Block structure of the mean field). The j-th block of $h^{(N)}$ is

$$\begin{aligned}&\rho(j)\,\mathbb{E}_{\mu_j}\big[\nabla_{\theta^{(j)}}\ell_j\big] \\ &\qquad + \sum_{i\ne j}\rho(i)\,\mathbb{E}_{\mu_i}\big[\mathbb{I}\{c(\Theta,s,i) = j\}\nabla_{\theta^{(j)}}\ell_j\big].\end{aligned}$$

Block j thus aggregates two losses with *different* targets: the GPI target when j is the current task, and its own greedy target, on the states where GPI selects it, when j is a prior. It is therefore not a multiple of the gradient of a single per-task loss. With E defined by (12), however, it equals $\nabla_{\theta^{(j)}}E$ on $\mathcal{U}$, which is all the proof of Theorem 1 requires.

APPENDIX B: EXPERIMENTAL DETAILS

B.1 Environments

Four Rooms [2] (discrete features): The agent navigates a four-room grid world to reach a fixed goal while collecting objects. Each object yields a reward only when it is first collected.

Tasks. The state concatenates one-hot grid coordinates with a binary inventory vector of collected objects. The feature vector $\phi(s, a, s') \in \{0, 1\}^{n_o+1}$ indicates whether an object of type $o \in \{1, \dots, n_o\}$ was collected or the goal was reached. Tasks are defined by reward weights $\mathbf{w}_j$ with $r = \phi^\top \mathbf{w}_j$, so that they assign different values to the objects.

Reacher [3] (continuous features): We use the PyBullet implementation of a two-joint robotic arm that must reach a target location. The state consists of the joint angles and velocities, and the actions are 9 discretized joint torques.

Tasks. Each task corresponds to a different target location. The features ϕ combine the state with a proximity measure, so that the reward is approximately linear in the features.

PointMaze [4] (continuous features): A point agent navigates a 2D maze to reach a goal. The state consists of the agent's observation and the goal coordinates, and the continuous action space is discretized into 9 actions by mapping $\{-1, 0, 1\}^2$ to control inputs.

Tasks. Each task is defined by a distinct goal location g_i, with dense reward

$$r_i(s, a, s') = \exp\left(-\|x(s') - g_i\|\right), \tag{25}$$

where $x(s')$ denotes the agent's position. We evaluate on layouts of increasing complexity (`UMaze`, `Medium`, `Large`). In every layout, episodes start in cell $(3, 3)$, and the 8 goals are cell $(1, 1)$ and its nearest free cells, so that neighboring tasks are related. Since `UMaze` has only 7 free cells, its eighth goal is the same cell as the seventh.

Linear successor features: For Reacher and PointMaze, whose rewards are linear in ϕ, we use the linear-SF parameterization $\psi_j(s, a) \in \mathbb{R}^{\dim \phi}$. The update carries over, with the sum over Φ in (8) replaced by the squared Euclidean norm of the vector residual $\phi(s, a, s') + \gamma(s')\psi_j(s', b) - \psi_j(s, a)$.

B.2 Hyperparameters

Table I lists the hyperparameters of the experiments of Section IX.

TABLE I: Experimental hyperparameters

Parameter	Four Rooms	Reacher	PointMaze
Steps per task	10,000	40,000	30,000
Number of tasks	6	4	8
Exploration (ϵ)	0.60	0.55	0.60
Batch size	512		
Replay buffer size	200,000		
Discount (γ)	0.95		
Episode horizon (T)	200		
Learning rate	0.001		
Reward-weight LR (α_R)	0.5		
Averaging samples (N)	5 (ablation: 5, 10, 20)		
Seeds	3		

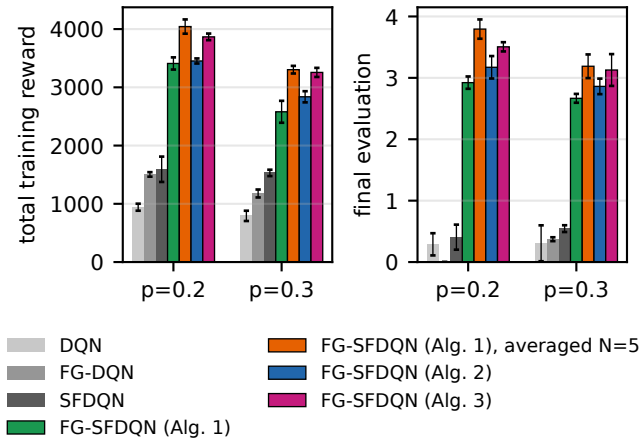
APPENDIX C: ADDITIONAL EXPERIMENTAL RESULTS

C.1 Training and evaluation totals

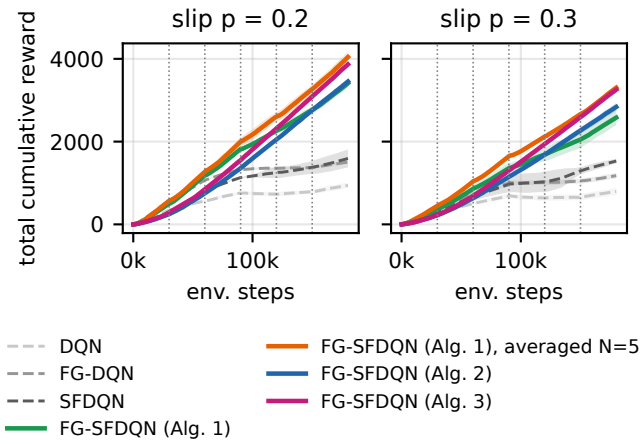
Tables II and III report the total training reward and the mean final evaluation return of all methods in the five domains of Section IX.

C.2 Slippery Four Rooms

Table IV and Fig. 3 report the totals of the slippery Four Rooms experiment of Section IX-C.3.



(a) Total training reward and final evaluation



(b) Total cumulative training reward over all tasks

Fig. 3: Slippery Four Rooms: totals (mean $\pm$ std over 3 seeds). Dotted lines in (b) mark the task switches of the sequential methods.

C.3 Slippery Reacher

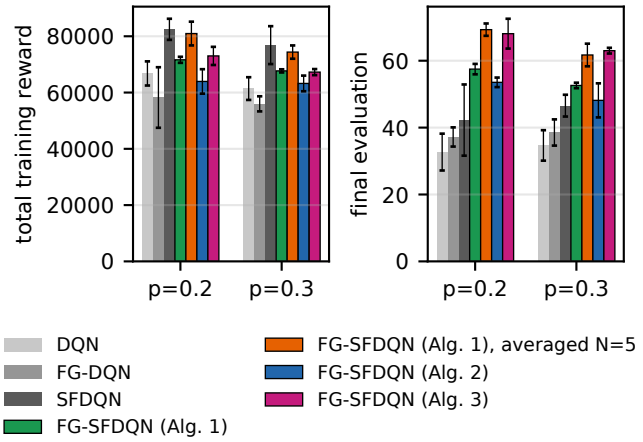
Table V and Fig. 4 report the totals of the slippery Reacher experiment of Section IX-C.3; the per-task results are in Fig. 2. The final evaluation is the mean return over 10 episodes of at most 100 steps on each of the 4 tasks.

TABLE II: Total cumulative training reward over all tasks (mean $\pm$ std over seeds).

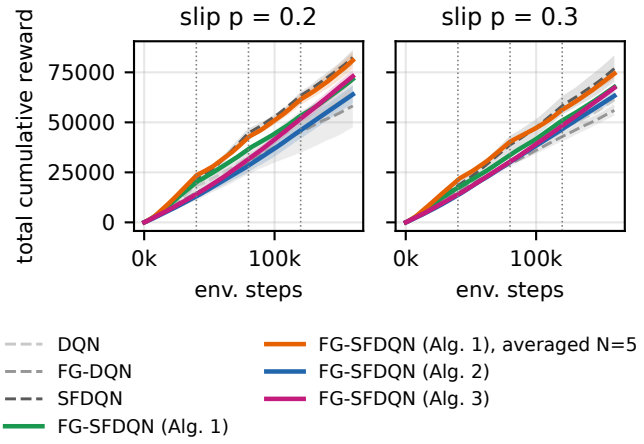
Method	Four Rooms	Reacher	PointMaze-UMaze	PointMaze-Medium	PointMaze-Large
DQN	341 ± 18	64996 ± 7318	61325 ± 14063	65897 ± 5462	58929 ± 1300
FG-DQN	611 ± 80	63744 ± 5407	63323 ± 324	57534 ± 2743	46333 ± 2793
SFDQN	500 ± 36	84135 ± 6232	74735 ± 5828	67948 ± 3551	54621 ± 5360
FG-SFDQN (Alg. 1)	$\mathbf{1365 \pm 185}$	$\mathbf{85328 \pm 4935}$	$\mathbf{84737 \pm 2500}$	$\mathbf{73746 \pm 3927}$	$\mathbf{61866 \pm 1750}$
FG-SFDQN (Alg. 2)	990 ± 96	79616 ± 4551	80877 ± 4144	68050 ± 1248	51609 ± 3526
FG-SFDQN (Alg. 3)	1081 ± 26	78684 ± 6687	82588 ± 1349	67217 ± 3439	51631 ± 2033

TABLE III: Final evaluation: mean return per episode, averaged over tasks (10 episodes per task, 100-step cap; mean $\pm$ std over seeds).

Method	Four Rooms	Reacher	PointMaze-UMaze	PointMaze-Medium	PointMaze-Large
DQN	0.03 ± 0.04	33.92 ± 5.08	18.98 ± 3.41	10.87 ± 1.53	15.01 ± 2.04
FG-DQN	0.02 ± 0.03	39.43 ± 1.53	13.81 ± 2.83	11.05 ± 4.20	11.63 ± 5.58
SFDQN	0.41 ± 0.15	38.68 ± 11.55	23.82 ± 3.30	21.00 ± 2.23	19.54 ± 0.70
FG-SFDQN (Alg. 1)	$\mathbf{3.38 \pm 0.03}$	$\mathbf{71.71 \pm 5.70}$	$\mathbf{29.54 \pm 0.37}$	$\mathbf{27.10 \pm 1.25}$	$\mathbf{27.24 \pm 0.71}$
FG-SFDQN (Alg. 2)	2.92 ± 0.35	68.54 ± 4.85	27.82 ± 0.65	24.75 ± 0.39	20.12 ± 1.81
FG-SFDQN (Alg. 3)	3.20 ± 0.31	68.05 ± 5.46	28.54 ± 0.94	25.92 ± 1.81	22.02 ± 3.36



(a) Total training reward and final evaluation (mean $\pm$ std over 3 seeds)



(b) Total cumulative training reward over all tasks

Fig. 4: Slippery Reacher totals: DQN, FG-DQN, SFDQN, Algorithm 1 and its averaged variant ($N = 5$), and Algorithms 2 and 3 (mean $\pm$ std over 3 seeds).

TABLE IV: Slippery Four Rooms: total training reward and final evaluation (mean $\pm$ std over 3 seeds)

p	Method	Training reward	Final evaluation
0.2	DQN	942 ± 60	0.29 ± 0.18
	FG-DQN	$1{,}505 \pm 39$	0.00 ± 0.00
	SFDQN	$1{,}593 \pm 218$	0.41 ± 0.20
	Alg. 1	$3{,}410 \pm 105$	2.92 ± 0.10
	Alg. 1, averaged ($N = 5$)	$\mathbf{4{,}043 \pm 122}$	$\mathbf{3.79 \pm 0.16}$
	Alg. 2	$3{,}452 \pm 44$	3.17 ± 0.18
	Alg. 3 ($N = 5$)	$3{,}866 \pm 57$	3.51 ± 0.07
0.3	DQN	793 ± 88	0.31 ± 0.29
	FG-DQN	$1{,}177 \pm 68$	0.37 ± 0.03
	SFDQN	$1{,}531 \pm 55$	0.54 ± 0.05
	Alg. 1	$2{,}580 \pm 188$	2.67 ± 0.07
	Alg. 1, averaged ($N = 5$)	$\mathbf{3{,}303 \pm 65}$	$\mathbf{3.19 \pm 0.19}$
	Alg. 2	$2{,}838 \pm 96$	2.86 ± 0.13
	Alg. 3 ($N = 5$)	$3{,}256 \pm 78$	3.13 ± 0.26

TABLE V: Slippery Reacher: total training reward and final evaluation (mean $\pm$ std over 3 seeds)

p	Method	Training reward	Final evaluation
0.2	DQN	$66{,}789 \pm 4{,}291$	32.7 ± 5.5
	FG-DQN	$58{,}239 \pm 10{,}760$	37.2 ± 2.9
	SFDQN	$\mathbf{82{,}470 \pm 3{,}750}$	42.3 ± 10.6
	Alg. 1	$71{,}634 \pm 1{,}072$	57.5 ± 1.6
	Alg. 1, averaged ($N = 5$)	$80{,}967 \pm 4{,}219$	$\mathbf{69.3 \pm 1.8}$
	Alg. 2	$63{,}949 \pm 4{,}347$	53.5 ± 1.4
	Alg. 3 ($N = 5$)	$73{,}018 \pm 3{,}223$	68.1 ± 4.5
0.3	DQN	$61{,}410 \pm 4{,}035$	34.7 ± 4.6
	FG-DQN	$55{,}988 \pm 2{,}682$	38.5 ± 3.9
	SFDQN	$\mathbf{76{,}853 \pm 6{,}729}$	46.6 ± 3.3
	Alg. 1	$67{,}698 \pm 561$	52.6 ± 0.8
	Alg. 1, averaged ($N = 5$)	$74{,}379 \pm 2{,}362$	61.7 ± 3.4
	Alg. 2	$63{,}241 \pm 2{,}779$	48.1 ± 5.1
	Alg. 3 ($N = 5$)	$67{,}267 \pm 1{,}077$	$\mathbf{63.0 \pm 0.9}$

C.4 Optimization diagnostics

During training, we periodically measure two quantities on replay batches using the online networks:

- the squared SF Bellman residual of the current task with the GPI target action, i.e., the per-sample objective of FG-SFRQL;
- the fraction of GPI target actions decided by a margin below 10^{-3}, i.e., near-ties.

Both are shown in Fig. 5, and the final residuals are listed in Table VI.

The full-gradient variants attain a much smaller Bellman residual than the semi-gradient SFDQN. At the end of training, the residual of Algorithm 1 is $46\times$ (Four Rooms), $22\times$ (Reacher) and 5.2–$6.8\times$ (all three PointMaze layouts) smaller than that of SFDQN. The randomized variants also improve on SFDQN, by 19–$20\times$ in Four Rooms, 17–$18\times$ in Reacher and 2.1–$2.4\times$ in PointMaze-UMaze and -Medium. In PointMaze-Large, Algorithm 2 improves on SFDQN by $1.2\times$, while both Algorithms 2 and 3 still surpass SFDQN in final evaluation return (20.12 and 22.02 vs. 19.54). For Algorithm 1, the residual spikes briefly at task switches when a new network starts training, and then decays rapidly.

This substantial reduction in Bellman error directly validates our theoretical formulation: differentiating through the bootstrapped target effectively suppresses the SF Bellman residual where semi-gradient updates stall, directly translating into higher final evaluation returns across all domains (Table III).

Near-ties in the GPI target action occur primarily at the beginning of training, when all value estimates are close to zero. By the end of training, they affect only 1.5–8.6% of the next states for the full-gradient variants, and ties in the choice of the GPI prior affect at most 1.5% of the states, providing strong empirical support for the tie-free condition of Theorem 1(ii).

C.5 Averaging ablation

Figure 6 compares Algorithm 1 with its averaged variant, which averages the Bellman target over $N \in \{5, 10, 20\}$ next states sampled from the same state-action pair, with per-sample GPI actions (Section V). The total training rewards (mean $\pm$ std over 3 seeds) are:

- Four Rooms: 1365 ± 185 for Algorithm 1, and 1333 ± 174, 1368 ± 36 and 1249 ± 22 for $N = 5, 10, 20$;
- Reacher: $85{,}328 \pm 4{,}935$ for Algorithm 1, and $85{,}251 \pm 3{,}143$, $82{,}845 \pm 3{,}529$ and $88{,}518 \pm 6{,}990$ for $N = 5, 10, 20$.

Consistent with Proposition 1, under deterministic transitions ($V = 0$) the mean returns across N all lie within one standard deviation of single-sample FG-SFDQN. Notably, larger sample sizes ($N = 10$ and $N = 20$) in Four Rooms reduce the standard deviation across seeds by up to $8\times$ (22 vs. 185), stabilizing the learning trajectory.

C.6 Computational overhead

Table VII reports the wall-clock time per training step on PointMaze-UMaze (8 tasks, batch size 512, NVIDIA RTX 4000 Ada GPU, Intel Core i9-14900KS CPU; 1,500 timed steps after a 512-step warm-up in each of 3 seeds). The SF methods are about $4\times$ (Algorithm 1, SFDQN) and $9\times$ (randomized variants) slower than the DQN baselines, since they evaluate the successor features of every policy for GPI. Within the SF family, replacing the semi-gradient update with the full gradient incurs no computational penalty: Algorithm 1 matches SFDQN in mean step time (4.34 vs. 4.47 ms) and achieves a lower median step time (2.67 vs. 4.19 ms). Furthermore, target averaging ($N = 5$) in Algorithm 1 adds modest overhead (5.11 ms mean), while still maintaining a lower median step time (3.85 ms) than SFDQN. The per-step runtime is primarily determined by the number of optimizer steps, one for the current task and one per distinct GPI-selected prior updated in the minibatch (correlation 0.97–1.00). While a full-gradient backward pass involves additional differentiation through the target, a likely factor contributing to Algorithm 1's lower median step time is that as policy preferences stabilize, fewer distinct priors may be triggered per batch. The randomized variants (Algorithms 2 and 3) take about twice as long (9.59 and 10.12 ms) because uniform task sampling interleaves transitions from all tasks across every batch.

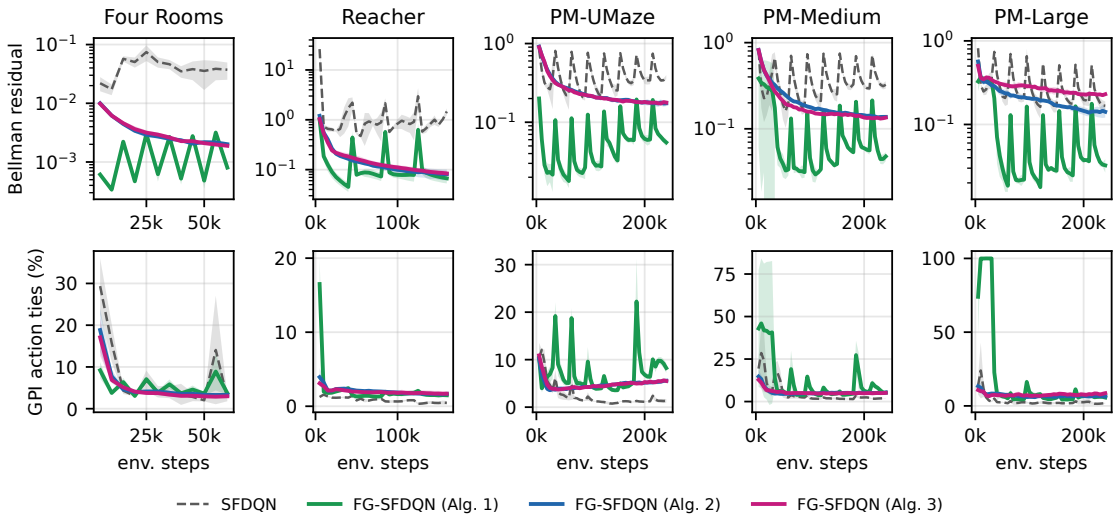
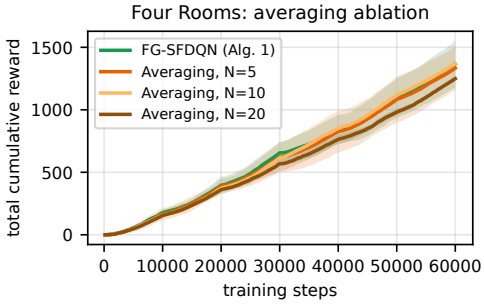


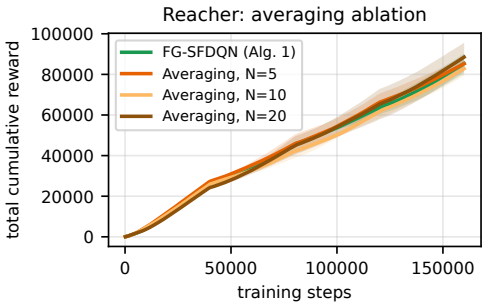
Fig. 5: Optimization diagnostics (mean $\pm$ std over 3 seeds). Top: squared SF Bellman residual of the current task (log scale). Bottom: percentage of GPI target actions decided by a margin below 10^{-3}.

TABLE VI: Final squared SF Bellman residual (mean over 3 seeds; lower is better)

Method	4R	Reacher	PM-U	PM-M	PM-L
SFDQN	0.0373	1.468	0.374	0.325	0.167
Alg. 1	**0.0008**	**0.067**	**0.055**	**0.048**	**0.032**
Alg. 2	0.0020	0.081	0.174	0.138	0.142
Alg. 3	0.0019	0.084	0.178	0.135	0.230



(a) Four Rooms



(b) Reacher

Fig. 6: Averaging ablation: total cumulative training reward of Algorithm 1 and its averaged variant with $N = 5, 10, 20$ next-state samples (mean $\pm$ std over 3 seeds).

TABLE VII: Per-step training time on PointMaze-UMaze

Algorithm	Mean (ms)	Median (ms)	Std (ms)
DQN	1.05	1.04	0.05
FG-DQN	1.19	1.18	0.04
SFDQN	4.47	4.19	1.22
FG-SFDQN (Alg. 1)	4.34	2.67	2.62
FG-SFDQN (Alg. 1, $N = 5$)	5.11	3.85	2.83
FG-SFDQN (Alg. 2)	9.59	9.61	0.44
FG-SFDQN (Alg. 3)	10.12	10.12	0.46

REFERENCES

[1] M. Benaïm, J. Hofbauer, and S. Sorin, "Stochastic approximations and differential inclusions," *SIAM Journal on Control and Optimization*, vol. 44, no. 1, pp. 328–348, 2005.

[2] M. Chevalier-Boisvert, B. Dai, M. Towers, R. Perez-Vicente, L. Willems, S. Lahlou, S. Pal, P. S. Castro, and J. Terry, "Minigrid & miniworld: Modular & customizable reinforcement learning environments for goal-oriented tasks," *Advances in neural information processing systems*, vol. 36, pp. 73 383–73 394, 2023.

[3] E. Coumans and Y. Bai, "Pybullet, a python module for physics simulation for games, robotics and machine learning," 2016.

[4] R. de Lazcano, K. Andreas, J. J. Tai, S. R. Lee, and J. Terry, "Gymnasium robotics," 2024. [Online]. Available: http://github.com/Farama-Foundation/Gymnasium-Robotics

APPENDIX D: ALGORITHMS

Algorithm 1 Joint Full-Gradient Model-Free SFRQL (FG-SFRQL), full pseudocode

Input: exploration rate ϵ; step sizes $\{\alpha_t\}$ for the ξ-functions; learning rate for reward models α_R
Input: features ϕ or Φ; optional: reward functions for tasks $\{\mathcal{R}_1, \mathcal{R}_2, \ldots, \mathcal{R}_{\text{num_tasks}}\}$

for $i = 1$ to num_tasks **do**
  if $\mathcal{R}_i$ not provided **then**
    initialize $\tilde{\mathcal{R}}_i$: $\theta_i^R \leftarrow$ small random initial values
  if $i = 1$ **then**
    initialize ξ_1: $\theta_1^\xi \leftarrow$ small random initial values
  else
    $\theta_i^\xi \leftarrow \theta_{i-1}^\xi$
  new_episode $\leftarrow$ **true**
  for $t = 1$ to num_steps **do**
    if new_episode **then**
      new_episode $\leftarrow$ **false**
      $s_t \leftarrow$ initial state
    $c \leftarrow \arg\max_{k\in\{1,\ldots,i\}} \max_a \sum_{\phi\in\Phi} \xi_k(s_t, a, \phi; \theta_k^\xi)\tilde{\mathcal{R}}_i(\phi)$ ▷ GPI prior at the current state
    With probability ϵ select a random action a_t, otherwise $a_t \leftarrow \arg\max_a \sum_{\phi\in\Phi} \xi_c(s_t, a, \phi; \theta_c^\xi)\tilde{\mathcal{R}}_i(\phi)$
    Take action a_t, observe reward r_t and next state s_{t+1}. Let $\phi_t = \phi(s_t, a_t, s_{t+1})$.
    if $\tilde{\mathcal{R}}_i$ not provided **then**
      Update θ_i^R using $\text{SGD}(\alpha_R)$ with $\mathcal{L}_R = (r_t - \tilde{\mathcal{R}}_i(\phi_t))^2$
    if s_{t+1} is a terminal state **then**
      $\gamma_t \leftarrow 0$; new_episode $\leftarrow$ **true**
    else
      $\gamma_t \leftarrow \gamma$
    ▷ GPI next action for task i
    $\hat{a}_{t+1} \leftarrow \arg\max_{a'} \max_{k\in\{1,\ldots,i\}} \sum_{\phi'\in\Phi} \xi_k(s_{t+1}, a', \phi'; \theta_k^\xi)\tilde{\mathcal{R}}_i(\phi')$
    ▷ Full-gradient update for policy i (the factor 2 of (10) is absorbed into α_t)
    $\nabla_{\theta_i^\xi}\mathcal{L}_t^{(i)} \leftarrow \sum_{\phi\in\Phi}\Big[\Big(\mathbb{I}(\phi=\phi_t) + \gamma_t\xi_i(s_{t+1}, \hat{a}_{t+1}, \phi; \theta_i^\xi)\Big) - \xi_i(s_t, a_t, \phi; \theta_i^\xi)\Big] \times \Big(\gamma_t\nabla_{\theta_i^\xi}\xi_i(s_{t+1}, \hat{a}_{t+1}, \phi; \theta_i^\xi) - \nabla_{\theta_i^\xi}\xi_i(s_t, a_t, \phi; \theta_i^\xi)\Big)$
    $\theta_i^\xi \leftarrow \theta_i^\xi - \alpha_t\nabla_{\theta_i^\xi}\mathcal{L}_t^{(i)}$
    if $c \neq i$ **then**
      ▷ Greedy next action for the executed policy c under its own reward
      $\hat{a}_{t+1}^c \leftarrow \arg\max_{a'} \sum_{\phi'\in\Phi} \xi_c(s_{t+1}, a', \phi'; \theta_c^\xi)\tilde{\mathcal{R}}_c(\phi')$
      ▷ Full-gradient update for policy c
      $\nabla_{\theta_c^\xi}\mathcal{L}_t^{(c)} \leftarrow \sum_{\phi\in\Phi}\big[\big(\mathbb{I}(\phi=\phi_t) + \gamma_t\xi_c(s_{t+1}, \hat{a}_{t+1}^c, \phi; \theta_c^\xi)\big) - \xi_c(s_t, a_t, \phi; \theta_c^\xi)\big] \times \Big(\gamma_t\nabla_{\theta_c^\xi}\xi_c(s_{t+1}, \hat{a}_{t+1}^c, \phi; \theta_c^\xi) - \nabla_{\theta_c^\xi}\xi_c(s_t, a_t, \phi; \theta_c^\xi)\Big)$
      $\theta_c^\xi \leftarrow \theta_c^\xi - \alpha_t\nabla_{\theta_c^\xi}\mathcal{L}_t^{(c)}$
    For all other $j \notin \{i, c\}$, $\theta_j^\xi \leftarrow \theta_j^\xi$.
    $s_t \leftarrow s_{t+1}$

Algorithm 2 Full-Gradient SFRQL with Randomized Task Sampling

Input: exploration rate ϵ; step sizes $\{\alpha_k\}$; reward learning rate α_R; task distribution ρ; pivot distributions $\{\mu_i\}$
Input: features Φ; set of tasks $\{1, \dots, m\}$; optional task rewards $\{\mathcal{R}_i\}$

Initialize $\{\theta_j^\xi\}_{j=1}^m$ (successor-feature parameters) and $\{\theta_j^R\}_{j=1}^m$ (reward models)

for $k = 0, 1, 2, \dots$ **do**
  Sample a task index $i \sim \rho$ ▷ current task
  Sample a pivot $(s, a) \sim \mu_i$ (generative model or replay buffer)
  Sample $s' \sim P(\cdot \mid s, a)$; observe reward r and feature $\phi_k = \phi(s, a, s')$
  if $\mathcal{R}_i$ not provided **then**
    Update reward model $\tilde{\mathcal{R}}_i$ using $\mathrm{SGD}(\alpha_R)$ on loss $(r - \tilde{\mathcal{R}}_i(\phi_k))^2$
  $\gamma_k \leftarrow 0$ if s' is terminal, else $\gamma_k \leftarrow \gamma$

  ▷ GPI prior at the pivot state
  $c \leftarrow \arg\max_{j \in \{1,\dots,m\}} \max_{a''} \sum_{\phi \in \Phi} \xi_j(s, a'', \phi; \theta_j^\xi)\, \tilde{\mathcal{R}}_i(\phi)$

  ▷ Update for current task i (GPI target action; the factor 2 of (10) is absorbed into α_k)
  $\hat{a}^i \leftarrow \arg\max_{a'} \max_{j \in \{1,\dots,m\}} \sum_{\phi' \in \Phi} \xi_j(s', a', \phi'; \theta_j^\xi) \tilde{\mathcal{R}}_i(\phi')$
  $\nabla_{\theta_i^\xi} \mathcal{L}_k^{(i)} \leftarrow \sum_{\phi \in \Phi} \Big[\mathbb{I}(\phi = \phi_k) + \gamma_k \xi_i(s', \hat{a}^i, \phi; \theta_i^\xi) - \xi_i(s, a, \phi; \theta_i^\xi)\Big] \times \Big(\gamma_k \nabla_{\theta_i^\xi} \xi_i(s', \hat{a}^i, \phi; \theta_i^\xi) - \nabla_{\theta_i^\xi} \xi_i(s, a, \phi; \theta_i^\xi)\Big)$

  if $c \neq i$ **then**
    ▷ Update for GPI-selected prior c (own greedy target action)
    $\hat{a}^c \leftarrow \arg\max_{a'} \sum_{\phi' \in \Phi} \xi_c(s', a', \phi'; \theta_c^\xi) \tilde{\mathcal{R}}_c(\phi')$
    $\nabla_{\theta_c^\xi} \mathcal{L}_k^{(c)} \leftarrow \sum_{\phi \in \Phi} \Big[\mathbb{I}(\phi = \phi_k) + \gamma_k \xi_c(s', \hat{a}^c, \phi; \theta_c^\xi) - \xi_c(s, a, \phi; \theta_c^\xi)\Big] \times \Big(\gamma_k \nabla_{\theta_c^\xi} \xi_c(s', \hat{a}^c, \phi; \theta_c^\xi) - \nabla_{\theta_c^\xi} \xi_c(s, a, \phi; \theta_c^\xi)\Big)$
    $\theta_c^\xi \leftarrow \theta_c^\xi - \alpha_k \nabla_{\theta_c^\xi} \mathcal{L}_k^{(c)}$
  $\theta_i^\xi \leftarrow \theta_i^\xi - \alpha_k \nabla_{\theta_i^\xi} \mathcal{L}_k^{(i)}$ ▷ all selections above use the pre-update Θ_k

  (If pivots come from a replay buffer: act ϵ-greedily w.r.t. GPI in task i and store the transition.)

Algorithm 3 Full-Gradient SFRQL with Randomized Task Sampling and Averaging

Input: exploration rate ϵ; step sizes $\{\alpha_k\}$; batch sizes $\{N_k\}$; reward learning rate α_R; task distribution ρ; pivot distributions $\{\mu_i\}$

Input: features Φ; set of tasks $\{1, \dots, m\}$; optional task rewards $\{\mathcal{R}_i\}$

Initialize $\{\theta_j^\xi\}_{j=1}^m$, $\{\theta_j^R\}_{j=1}^m$ (and replay buffer $\mathcal{D} = \emptyset$ if used)

for $k = 0, 1, 2, \dots$ **do**

  Sample task $i \sim \rho$ and pivot $(s, a) \sim \mu_i$; let $N \leftarrow N_k$

  Draw N next states $\mathcal{K} = \{s'_p\}_{p=1}^N$ i.i.d. from $P(\cdot \mid s, a)$ (or from transitions in $\mathcal{D}$ that start at (s, a))

  if fewer than N transitions are available **then continue**

  $\phi_p \leftarrow \phi(s, a, s'_p)$, $\gamma_p \leftarrow \gamma(s'_p)$ for $p = 1, \dots, N$

  if $\mathcal{R}_i$ not provided **then**

    Update reward model $\tilde{\mathcal{R}}_i$ on the observed rewards using $\mathrm{SGD}(\alpha_R)$, as in Algorithm 2

  ▷ GPI prior at the pivot and per-sample greedy target actions

  $c \leftarrow \arg\max_j \max_{a''} \sum_\phi \xi_j(s, a'', \phi; \theta_j^\xi) \tilde{\mathcal{R}}_i(\phi)$

  $b_{i,p} \leftarrow \arg\max_{a'} \max_j \sum_{\phi'} \xi_j(s'_p, a', \phi'; \theta_j^\xi) \tilde{\mathcal{R}}_i(\phi')$ for each p

  $b_{c,p} \leftarrow \arg\max_{a'} \sum_{\phi'} \xi_c(s'_p, a', \phi'; \theta_c^\xi) \tilde{\mathcal{R}}_c(\phi')$ for each p (if $c \neq i$)

  ▷ Averaged residuals and feature gradients, Eqs. (8)–(10), for $j \in \{i, c\}$; the factor 2 is absorbed into α_k

  $\bar{\delta}_j(\phi) \leftarrow \frac{1}{N} \sum_{p=1}^N \left[\mathbb{I}(\phi = \phi_p) + \gamma_p \, \xi_j(s'_p, b_{j,p}, \phi; \theta_j^\xi)\right] - \xi_j(s, a, \phi; \theta_j^\xi)$ for all $\phi \in \Phi$

  $\bar{D}_j(\phi) \leftarrow \frac{1}{N} \sum_{p=1}^N \gamma_p \nabla \xi_j(s'_p, b_{j,p}, \phi; \theta_j^\xi) - \nabla \xi_j(s, a, \phi; \theta_j^\xi)$ for all $\phi \in \Phi$

  $\theta_i^\xi \leftarrow \theta_i^\xi - \alpha_k \sum_\phi \bar{\delta}_i(\phi) \, \bar{D}_i(\phi)$ ▷ current task

  if $c \neq i$ **then**

    $\theta_c^\xi \leftarrow \theta_c^\xi - \alpha_k \sum_\phi \bar{\delta}_c(\phi) \, \bar{D}_c(\phi)$ ▷ GPI-selected prior

  (If a replay buffer is used: act ϵ-greedily w.r.t. GPI in task i and store (s, a, r, s', ϕ, i) in $\mathcal{D}$.)